
Advanced Data Mining

Frequent itemset mining Techniques

Introduction

- ❑ Many retail stores collect data about customers.
- ❑ e.g. customer transactions
- ❑ Need to analyze this data to understand customer behavior
- ❑ Why?
 - for marketing purposes,
 - inventory management,
 - customer relationship management

Introduction

Discovering patterns and associations

- ❑ Discovering interesting relationships hidden in large databases.
 - ❑ e.g. **beer and diapers are often sold together**
 - ❑ **pattern mining** is a fundamental data mining problem with many applications in various fields.
 - ❑ Introduced by Agrawal (1993).
 - ❑ Many extensions of this problem to discover patterns in graphs, sequences, and other kinds of data.
-

Numerous applications

Frequent itemset mining has numerous applications.

- medical applications,
- chemistry,
- biology,
- e-learning,
- etc.

Definitions

Let $I = \{I_1, I_2, \dots, I_m\}$ be the set of **items** (products) sold in a retail store.

For example:

$I = \{\text{pasta, lemon, bread, orange, cake}\}$

Definitions

❑ Let $I = \{I_1, I_2, \dots, I_m\}$ be the set of **items** (products) sold in a retail store.

For example: $I = \{\text{pasta, lemon, bread, orange, cake}\}$

❑ A **transaction database** D is a set of **transactions**.

$D = \{T_1, T_2, \dots, T_r\}$ such that $T_a \subseteq I$ ($1 \leq a \leq r$).

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

Each transaction has a unique identifier called its *Transaction ID (TID)*.

e.g. the transaction ID of T_4 is 4.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

A transaction is a set of items (an *itemset*).

e.g. $T2 = \{\text{pasta, lemon}\}$

An **item** (a symbol) may not appear or appear once in each transaction. Each transaction is unordered.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

A transaction database can be viewed as a binary matrix:

Transaction	pasta	lemon	bread	orange	cake
T1	1	1	1	1	0
T2	1	1	0	0	0
T3	1	0	0	1	1
T4	1	1	0	1	1

- Asymmetrical binary attributes (because 1 is more important than 0)
 - There is no information about purchase quantities and prices.
-

Definitions

Let **I** be the set of all items:

$I = \{\text{pasta, lemon, bread, orange, cake}\}$

There are $2^{|I|} - 1 = 2^5 - 1 = 31$ subsets :

$\{\text{pasta}\}, \{\text{lemon}\}, \{\text{bread}\}, \{\text{orange}\}, \{\text{cake}\}$

$\{\text{pasta, lemon}\}, \{\text{pasta, bread}\}, \{\text{pasta, orange}\}, \{\text{pasta, cake}\}, \{\text{lemon, bread}\},$
 $\{\text{lemon, orange}\},$

$\{\text{lemon, cake}\}, \{\text{bread, orange}\}, \{\text{bread, cake}\}$

...

$\{\text{pasta, lemon, bread, orange, cake}\}$

Definitions

An **itemset** is said to be **of size k** , if it contains k items.

Itemsets of size 1:

{pasta}, {lemon}, {bread}, {orange}, {cake}

Itemsets of size 2:

{pasta, lemon}, {pasta, bread}, {pasta, orange}, {pasta, cake}, {lemon, bread},
{lemon orange}, ...

Definitions

The **support (frequency)** of an itemset **X** is the number of transactions that contains **X**.

$$\text{sup}(X) = |\{T \mid X \subseteq T \wedge T \in D\}|$$

For example: The support of {pasta, orange} is 3.

which is written as: $\text{sup}(\{\text{pasta}, \text{orange}\}) = 3$

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

The **support of an itemset X** can also be written as a ratio (*absolute support*).

Example: The support of {pasta, orange} is 75% because it appears in 3 out of 4 transactions.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

The problem of frequent itemset mining

- Let there be a numerical value *minsup*, set by the user.
- Frequent itemset mining (FIM) consists of enumerating all **frequent itemsets**, that is itemsets having a support greater or equal to *minsup*.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Example

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange cake}

For *minsup* = 2, the **frequent itemsets** are:

{lemon}, {pasta}, {orange}, {cake}, {lemon, pasta}, {lemon, orange}, {pasta, orange}, {pasta, cake}, {orange, cake}, {lemon, pasta, orange}

For the user, choosing a high *minsup* value,

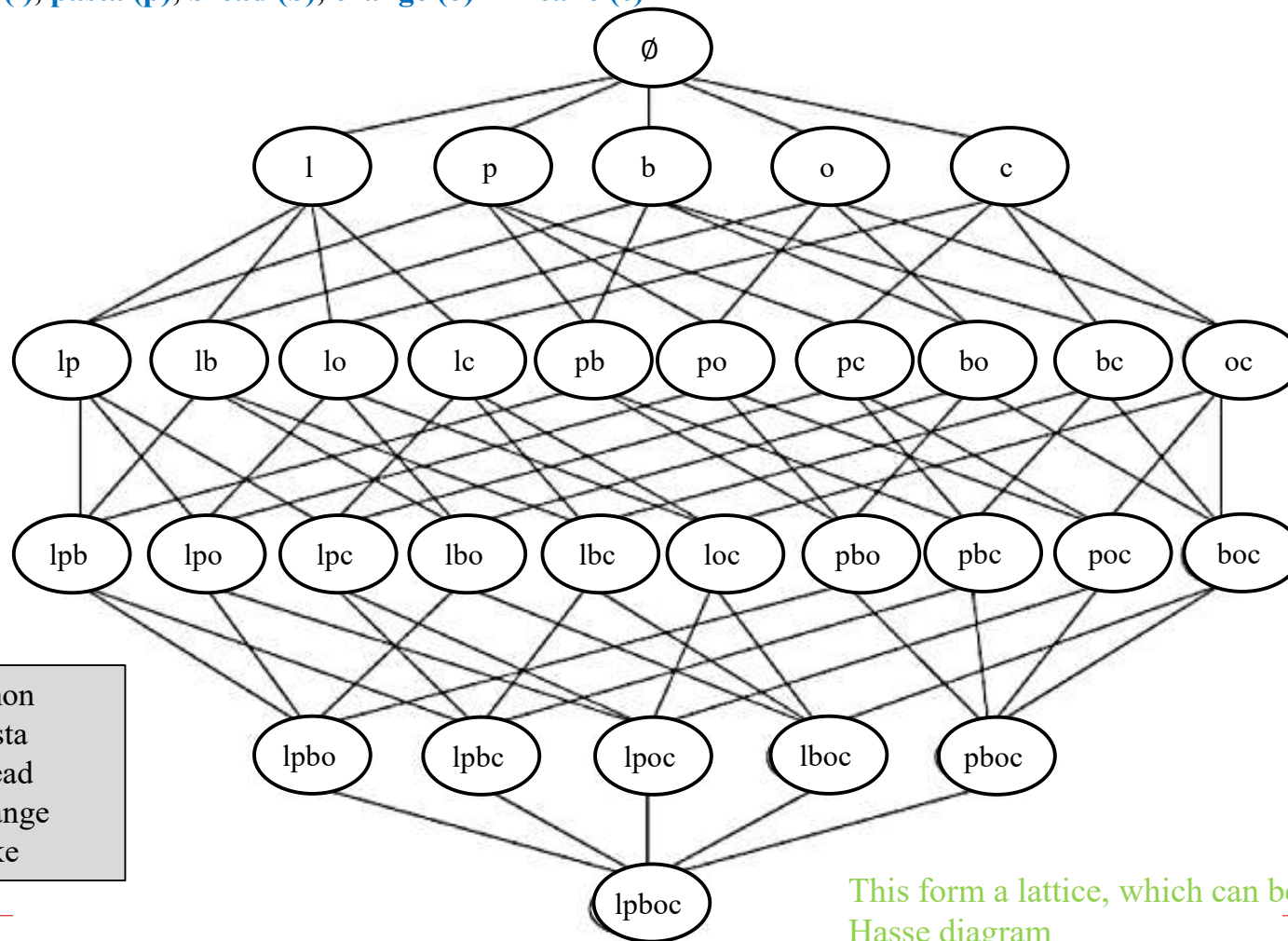
- ☐ will reduce the number of frequent itemsets,
- ☐ will increase the speed and decrease the memory required for finding the frequent itemsets

Naïve approach

- If there are n items in a database, there are $2^n - 1$ itemsets may be frequent.
- **Naïve approach:** count the support of all these itemsets.
- To do that, we would need to read each transaction in the database to count the support of each itemset.
- This would be inefficient:
 - need to perform too many comparisons
 - requires too much memory

Search space

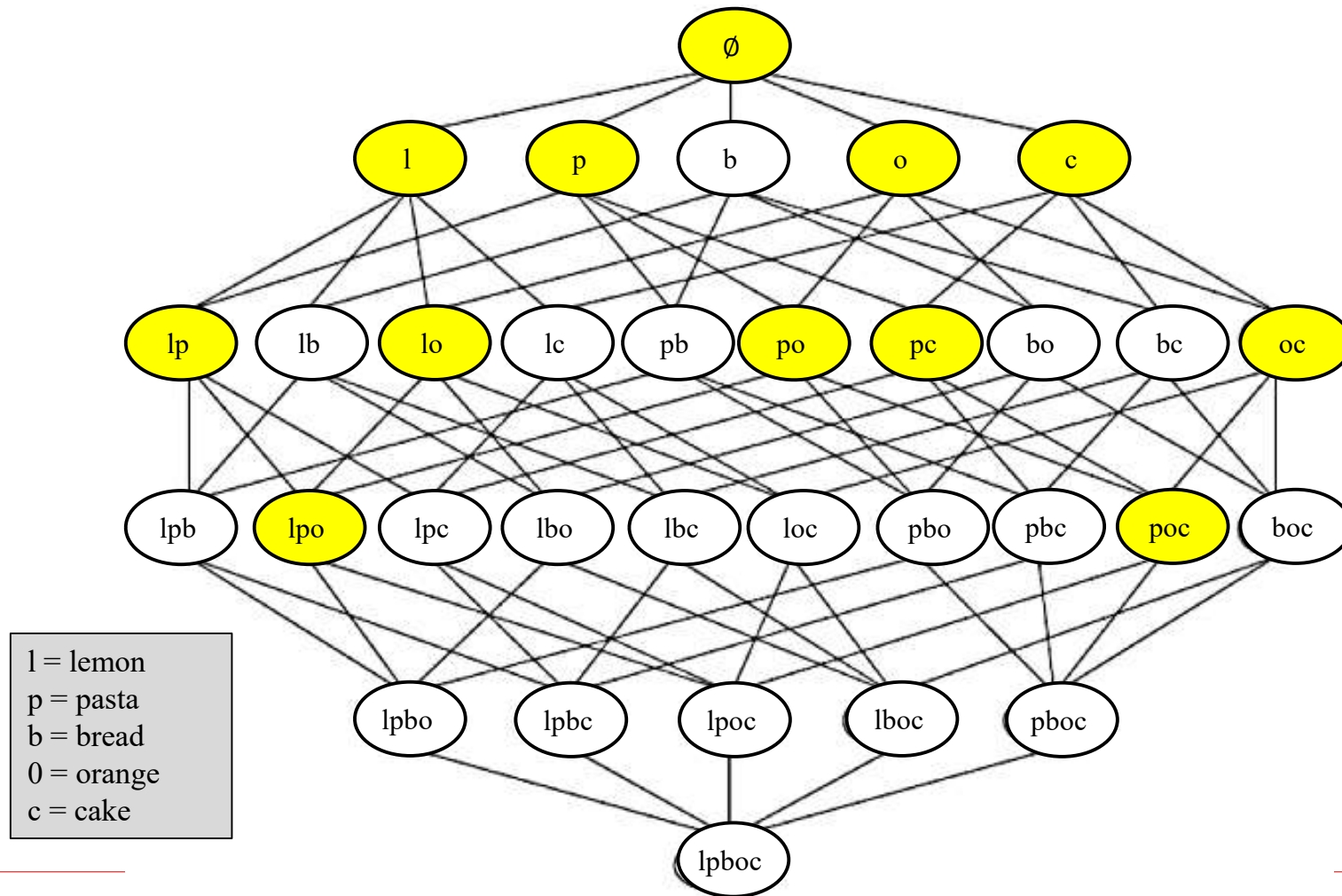
This is all the itemsets that can be formed with the items
lemon (l), pasta (p), bread (b), orange (o) and cake (c)

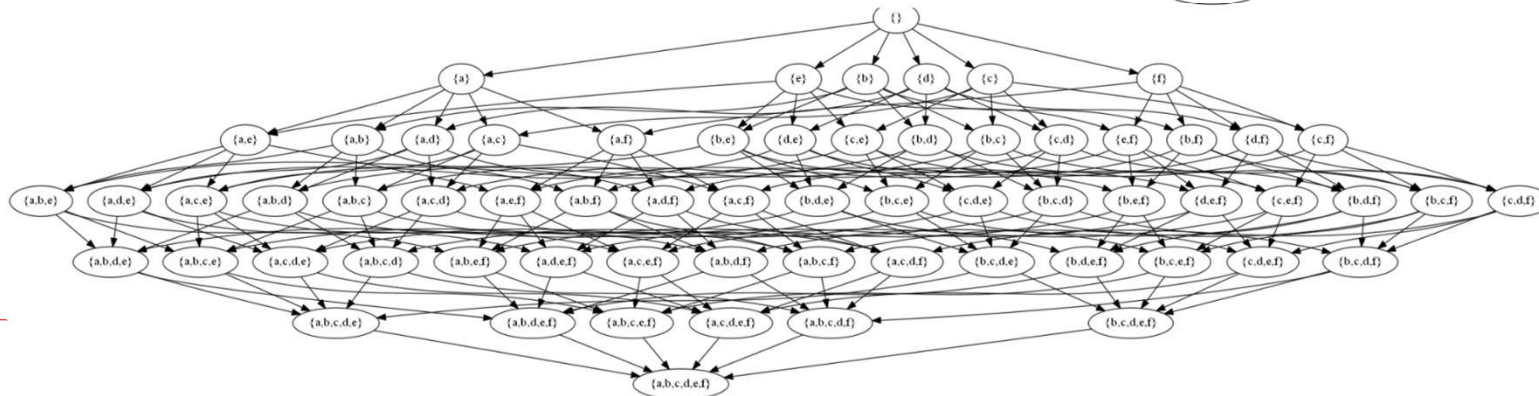
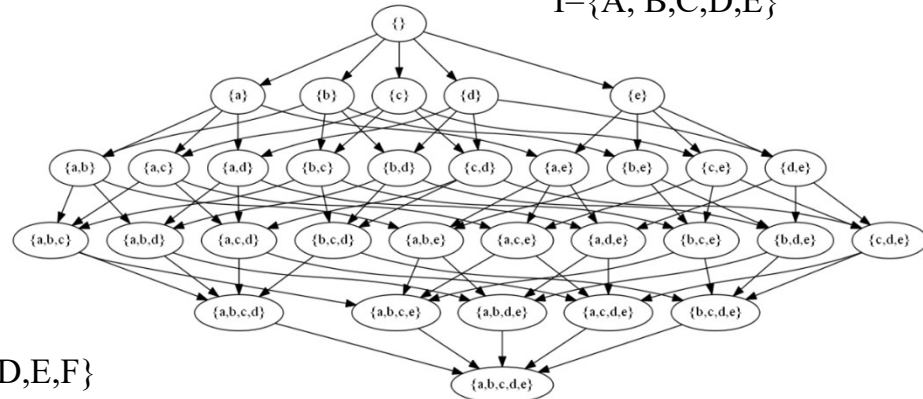
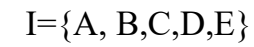
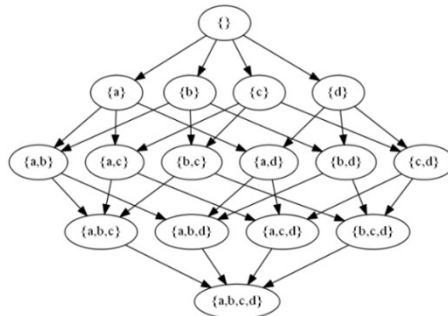
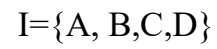
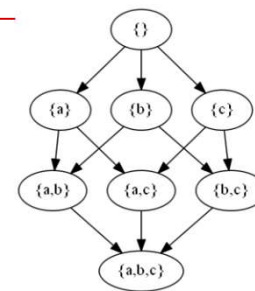
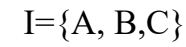
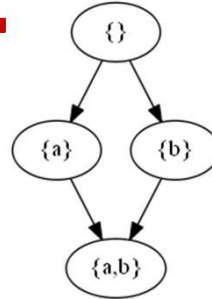
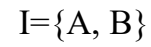
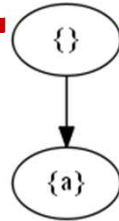
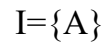


This form a lattice, which can be viewed as a
Hasse diagram

Search space

If **minsup** = 2, the frequent itemsets are (in yellow):





How to find the frequent itemsets?

Two challenges:

- ☐ How to count the support of itemsets in an efficient way (not spend too much time or memory)?
- ☐ How to reduce the search space (we do not want to consider all the possibilities)?

Several algorithms

- Algorithms:
 - **Apriori**, AprioriTID
 - Eclat
 - **FPGrowth**
 - **Hmine**
 - **LCM**, ...
 - ...
 - Moreover, numerous extensions of the FIM problem: uncertain data, fuzzy data, purchase quantities, profit, weight, time, rare itemsets, closed itemsets, etc.
-

The APRIORI algorithm (Agrawal & Srikant, 1993/1994)

*R. Agrawal and R. Srikant. Fast algorithms for mining
association rules in large databases. Research Report
RJ 9839, IBM Almaden Research Center, San Jose,
California, June 1994.*

Introduction

Apriori is a famous algorithm

- ❑ which is not the most efficient algorithm,
- ❑ but has inspired many other algorithms!
- ❑ has been applied in many fields,
- ❑ has been adapted for many other similar problems.

Apriori is based on two important properties

Apriori property: Let there be two itemsets X and Y .
If $X \subset Y$, the support of Y is less than or equal to the support of X .

Example:

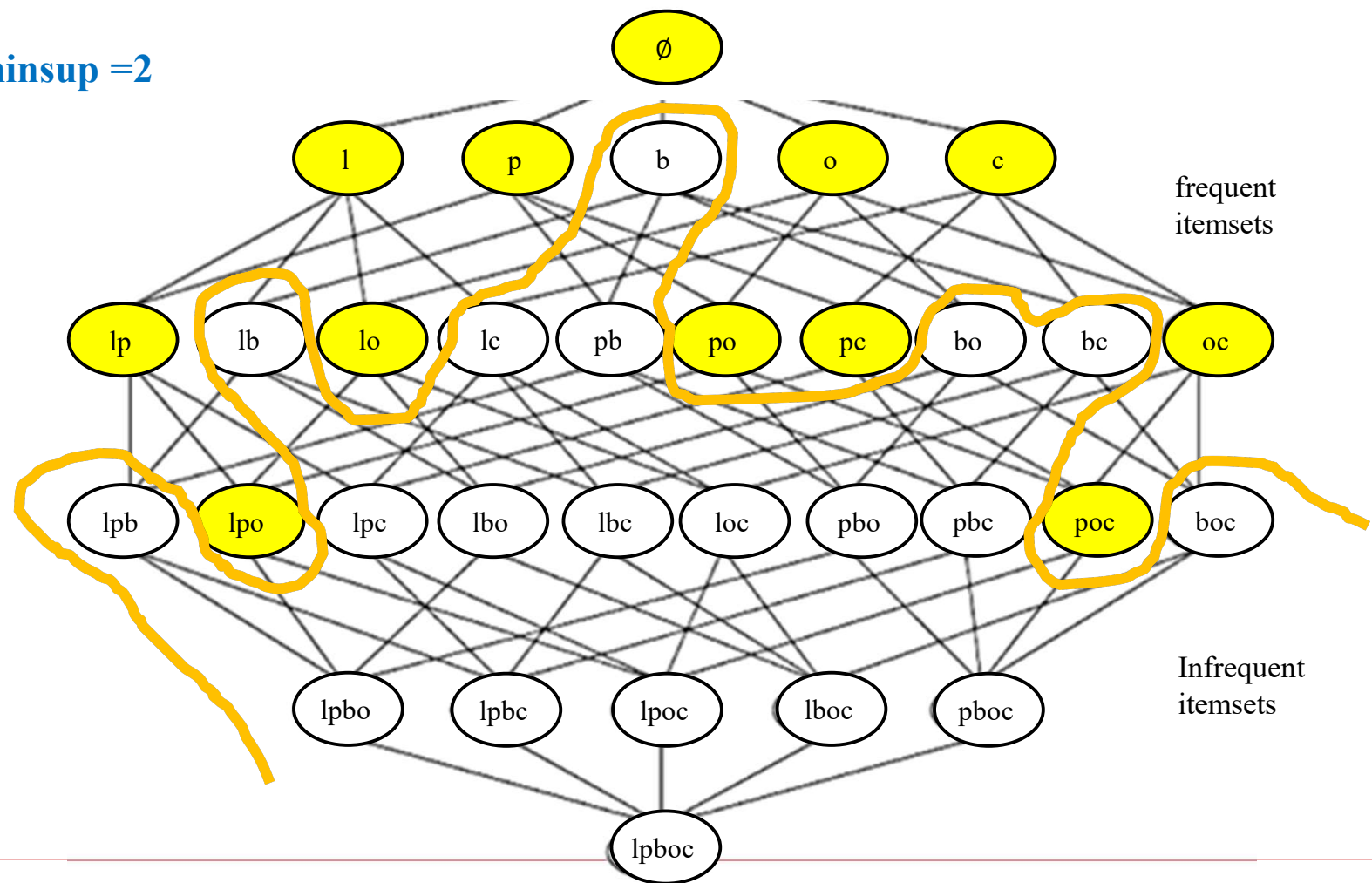
- The support of {pasta} is 4
- The support of {pasta, lemon} is 3
- The support of {pasta, lemon, orange} is 2

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

(support is anti-monotonic)

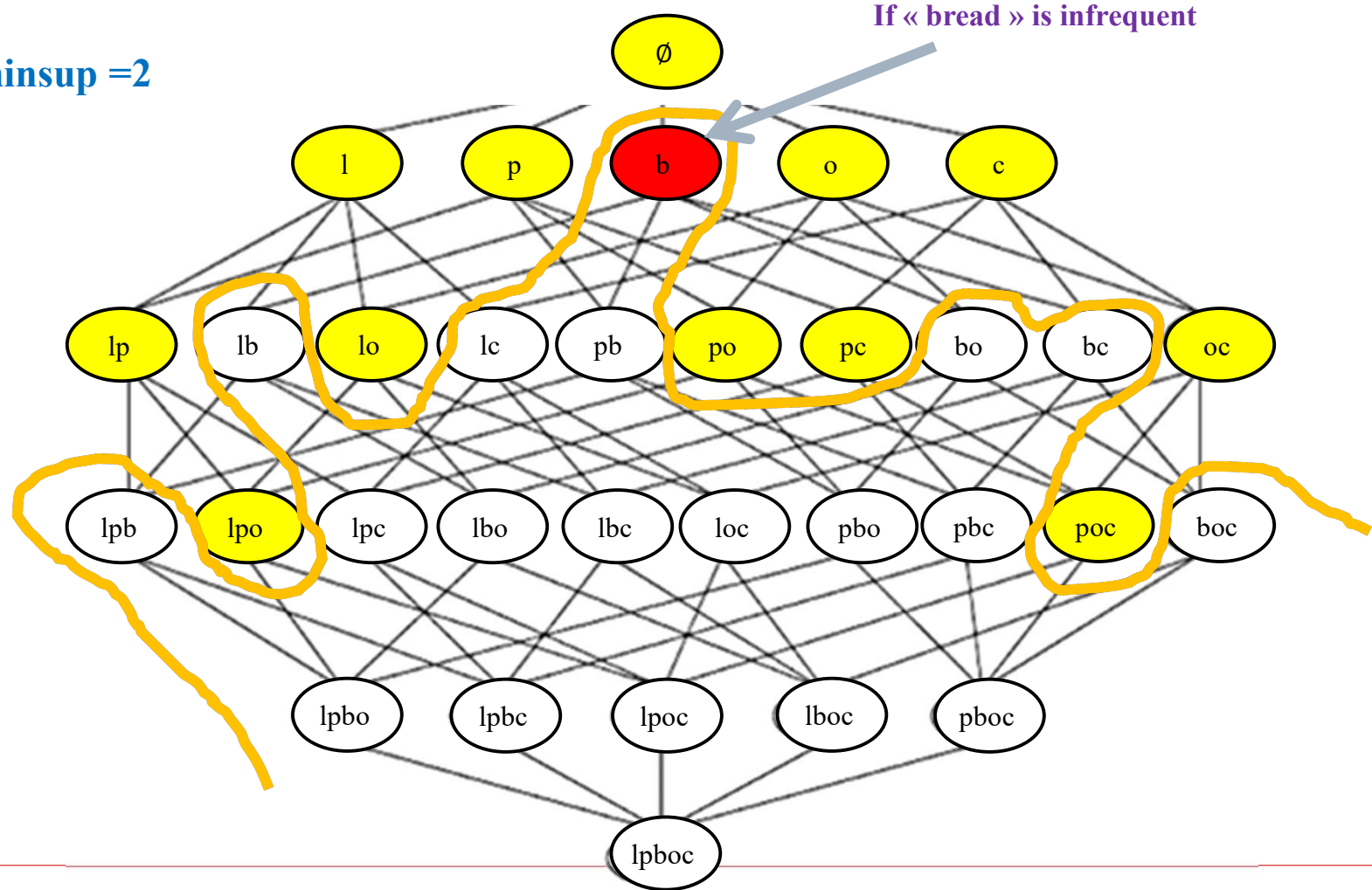
Illustration

minsup = 2



This property is useful to reduce the search space. **Example:**

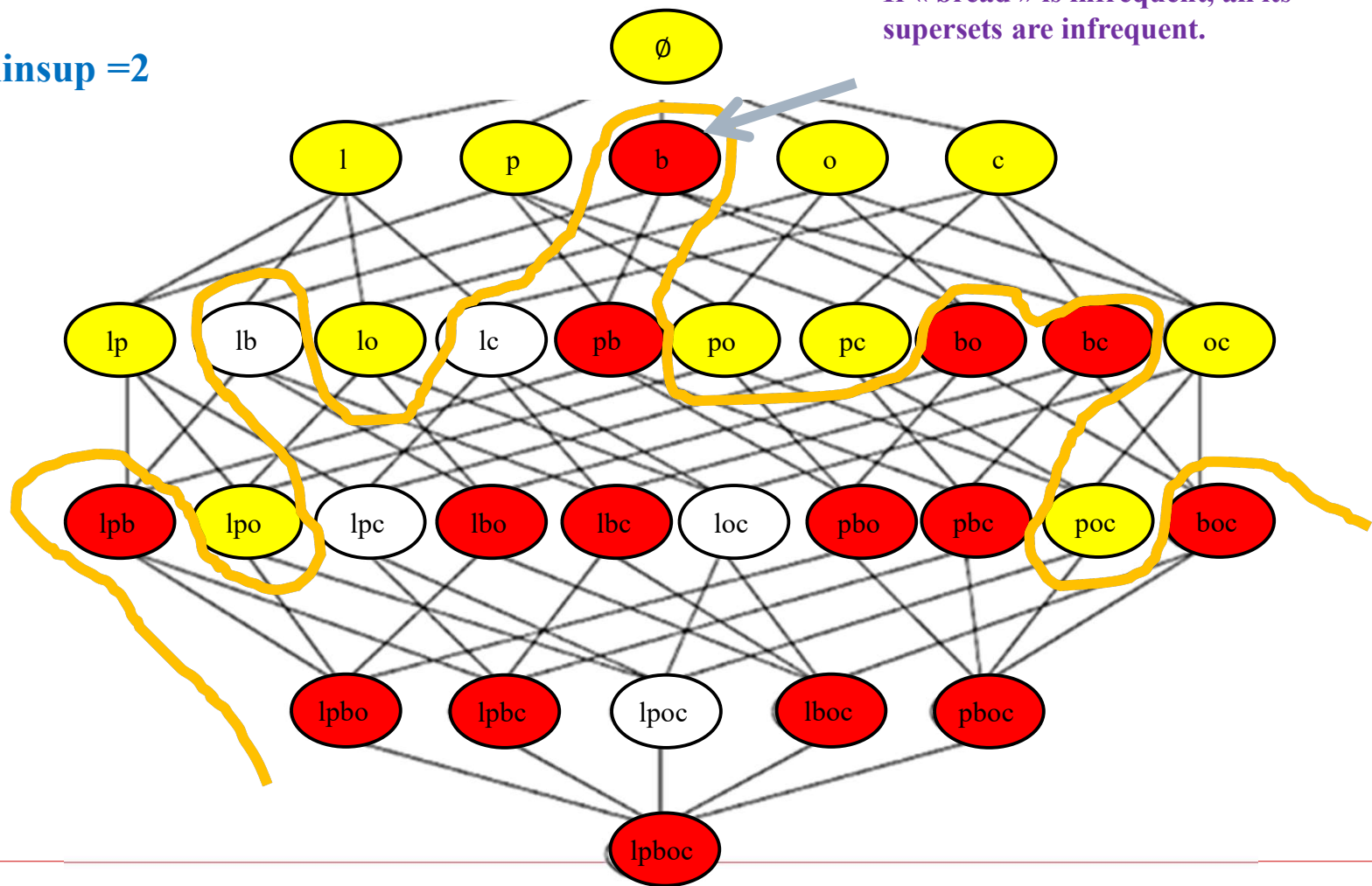
minsup = 2



This property is useful to reduce the search space. **Example:**

minsup = 2

If « bread » is infrequent, all its
supersets are infrequent.



Property 2: Let there be an itemset Y .

If there exists an itemset $X \subset Y$ such that X is infrequent, then Y is infrequent.

Example:

- Consider $\{\text{bread, lemon}\}$.
- If we know that $\{\text{bread}\}$ is infrequent, then we can infer that $\{\text{bread, lemon}\}$ is also infrequent.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

The Apriori algorithm

- I will now explain how the Apriori algorithm works
- **Input:**
 - *minsup*
 - a transactional database
- **Output:**
 - all the frequent itemsets

Consider *minsup* = 2.

Apriori vs the naïve algorithm

- The Apriori property can considerably reduce the number of itemsets to be considered.
- In the previous example:
 - Naïve approach:
 $2^5 - 1 = 31$ itemsets are considered
 - By using the Apriori property:
18 itemsets are considered

How to evaluate this type of algorithms?

- ☐ Execution time,
 - ☐ Memory used,
 - ☐ *Scalability*: how the performance is influenced by the number of transactions
 - ☐ Performance on different types of data:
 - real data,
 - synthetic (fake) data,
 - dense vs sparse data,...
 - ☐ ...
-

Performance of Apriori

The performance of Apriori depends on several factors:

- ❑ the *minsup* parameter: the more it is set low, the larger the search space and the number of itemsets will be.
 - ❑ the number of items,
 - ❑ the number of transactions,
 - ❑ The average transaction length.
-

Problems of Apriori

- ☐ can generate numerous candidates
 - ☐ requires to scan the database numerous times.
 - ☐ candidates may not exist in the database.
 - ☐ ...
-

Factors Affecting Complexity of Apriori

- ❑ Choice of minimum support threshold
 - lowering support threshold results in more frequent itemsets
 - this may increase number of candidates and max length of frequent itemsets
- ❑ Dimensionality (number of items) of the data set
 - More space is needed to store support count of itemsets
 - if number of frequent itemsets also increases, both computation and I/O costs may also increase
- ❑ Size of database
 - run time of algorithm increases with number of transactions
- ❑ Average transaction width
 - transaction width increases the max length of frequent itemsets
 - number of subsets in a transaction increases with its width, increasing computation time for support counting
- ❑ Solution: Concise Itemset Representation
 - ➔ Mine *closed patterns* and *max-patterns*

A few optimizations for the APRIORI algorithm

This is an advanced topic

Optimization 1

In terms of data structure:

- ❑ Store all items as integers:
e.g. 1= pasta, 2= orange, 3= bread...

 - ❑ Why?
 - it is faster to compare two integers than to compare two character strings,
 - requires less memory.
-

Optimization 2

To reduce the time required to calculate the support of itemsets

Transaction	Items appearing in the transaction	
T2	{pasta, lemon}	sort
T3	{pasta, orange, cake}	transactions
T4	{pasta, lemon, orange, cake}	by ascending
T1	{pasta, lemon, bread, orange}	length

- To calculate the support of an itemset of size k , only the transactions of size $\geq k$ are used.

Optimization 3

To reduce the time required to calculate the support of itemsets

- Replace all identical transactions by a single transactions with a weight.

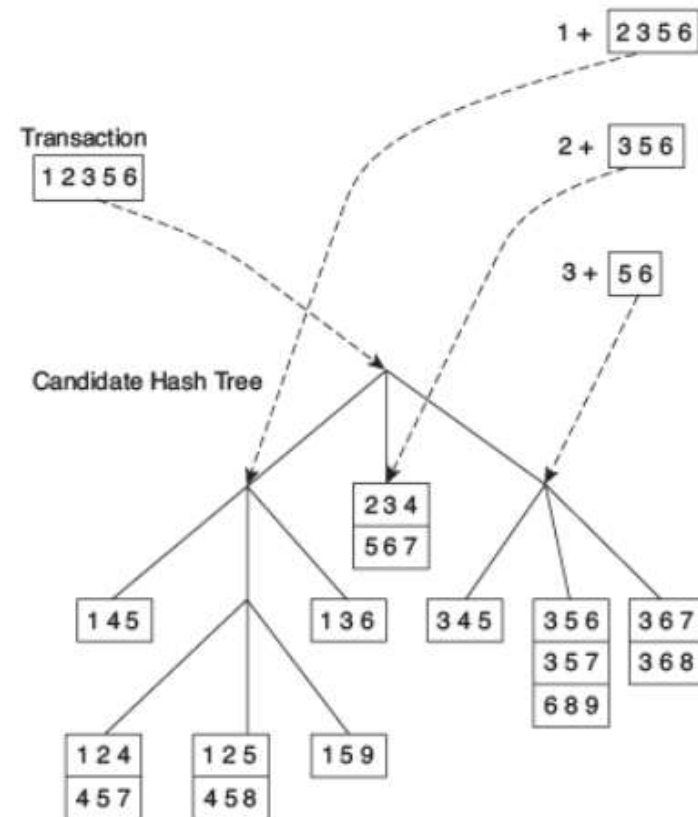
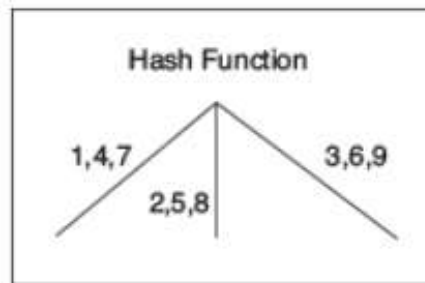
Optimization 4

To reduce the time require to calculate the support of itemsets:

- ❑ Sort items in transactions according to a total order (e.g. [alphabetical order](#)).
- ❑ Utilize binary search to quickly check if an item appears in a transaction.

Optimization 5

- Store candidates in a hash tree
- To calculate the support of candidates
 - Calculate a hash value based on a transaction to determine if candidates are contained in the transaction.



Other optimizations

- Sampling and partitioning

AprioriTID: a variation

AprioriTID:

- Annotate each itemset with the ids of transactions that contain it,
- use the intersection ($\cap$) to calculate the support of itemsets instead of reading the database.

Example →

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}



item	transactions containing the item
pasta	T1, T2, T3, T4
lemon	T1, T2, T3
bread	T1
orange	T1, T3, T4
cake	T3, T4

item	transactions containing the item
pasta	T1, T2, T3, T4
lemon	T1, T2, T4
bread	T1
orange	T1, T3, T4
cake	T3, T4

Example: calculating the support of {pasta, lemon} :

$$\begin{aligned}
 & \text{transactions}(\{\text{pasta}\}) \cap \text{transactions}(\{\text{lemon}\}) \\
 &= \{\text{T1, T2, T3, T4}\} \cap \{\text{T1, T2, T4}\} \\
 &= \{\text{T1, T2, T4}\}
 \end{aligned}$$

Thus {pasta, lemon} has a support of 3

AprioriTID_Bitset

AprioriTID_bitset:

- ❑ Same idea, except that bit vectors are used instead of lists of ids.
- ❑ This allows to calculate the intersection using the **Logical_AND**, which is often very fast.

Example →

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}



item	transactions containing the item
pasta	1111 (representing T1, T2, T3, T4)
lemon	1101
bread	1000
orange	1011
cake	0011

item	transactions containing the item
pasta	1111
lemon	1101
bread	1000
orange	1011
cake	0011

Example: Calculate the support of {pasta, lemon} :

$$\begin{aligned}
 &\text{transactions}(\{\text{pasta}\}) \cap \text{transactions}(\{\text{lemon}\}) \\
 &= 1111 \text{ LOGICAL_AND } 1101 \\
 &= 1101
 \end{aligned}$$

Thus {pasta, lemon} has a support of 3

The ECLAT algorithm

Mohammed Javeed Zaki: Scalable Algorithms for Association Mining. IEEE Trans. Knowl. Data Eng. 12(3): 372-390 (2000)

Eclat (Zaki, 2000)

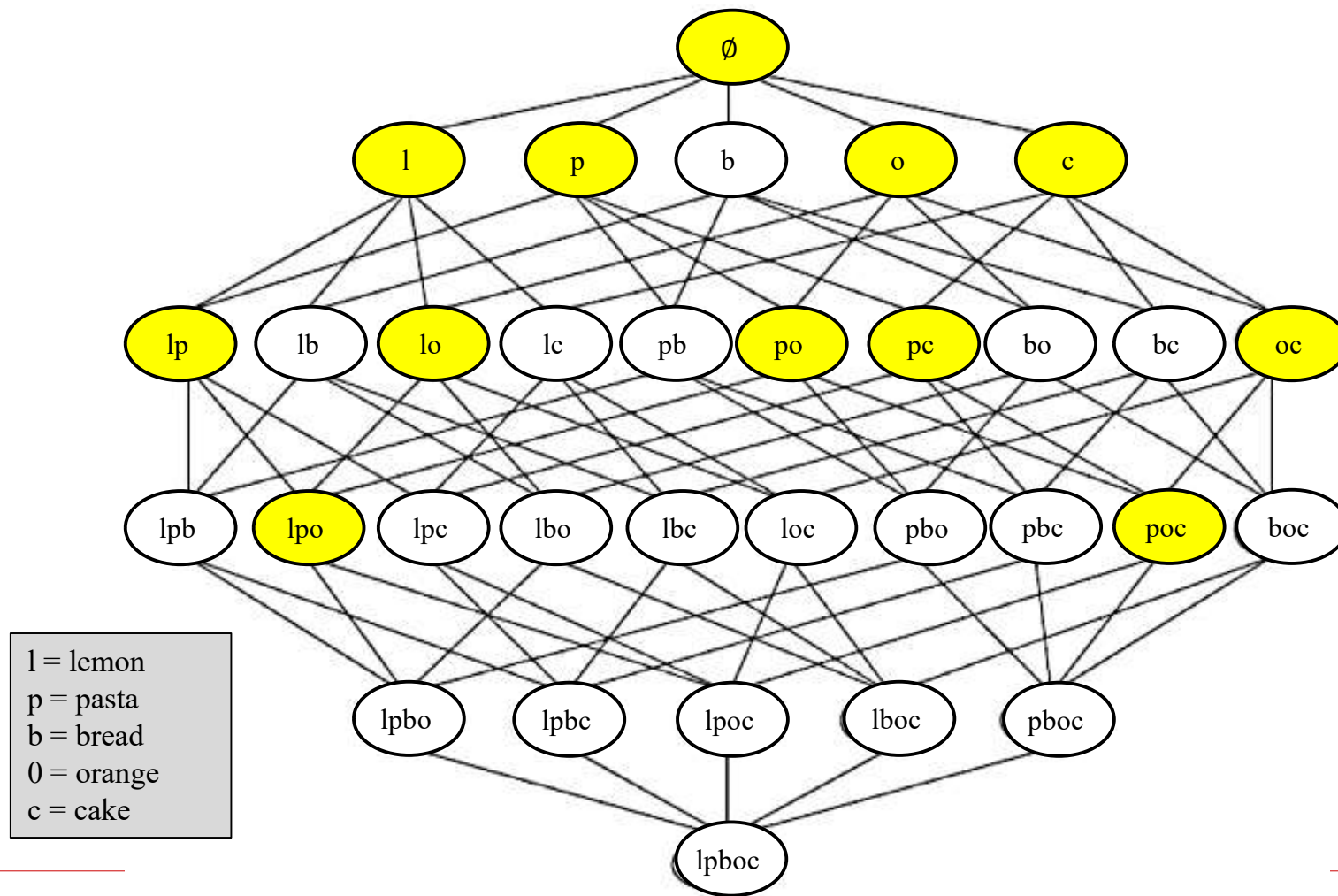
- ❑ ECLAT (**E**quivalence **CL**ass **T**ransformation)
 - ❑ An algorithm that is generally faster than **Apriori**.
 - ❑ Utilize a **depth-first search** (contrarily to Apriori/AprioriTID).
 - ❑ Utilize a *vertical database* (as AprioriTID)
 - ❑ Utilize the concept of **equivalence classes of itemsets** sharing the same prefix.
-

Definitions

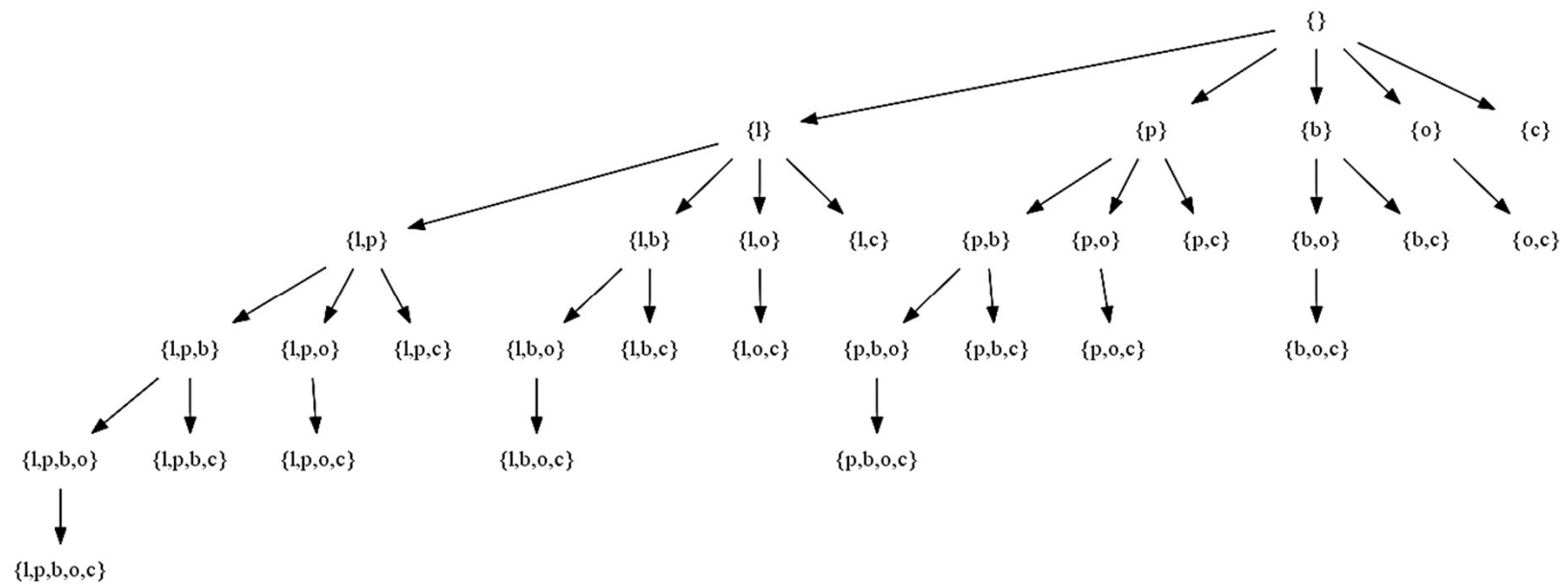
Total order

- Without loss of generality, we suppose that all transactions and itemsets are sorted according to a **total order** $<$.
- This total order $<$ can for example be the alphabetical order.
- e.g.
pasta $<$ lemon $<$ bread $<$ orange $<$ cake

Search space



Search space



l = lemon
p = pasta
b = bread
o = orange
c = cake

The search space can be visualized as a **set enumeration tree**
- Rymon, 1992

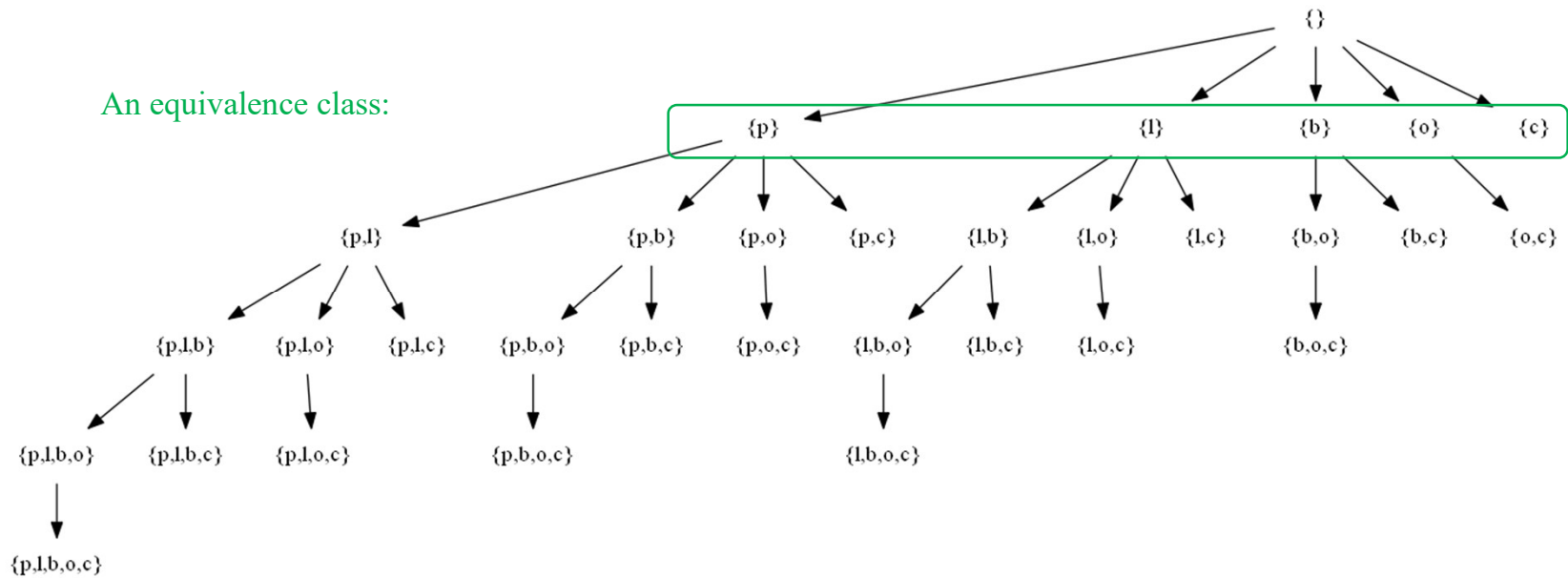
Definitions

Equivalence class

- Let there be two itemsets **X** and **Y** of size k .
- **X** and **Y** belong to the same equivalence class if the $k - 1$ first items of **X** and **Y** are the same according to the total order.
- e.g. An equivalence class:
 - {pasta, lemon, bread},
 - {pasta, lemon, orange},
 - {pasta, lemon, cake}

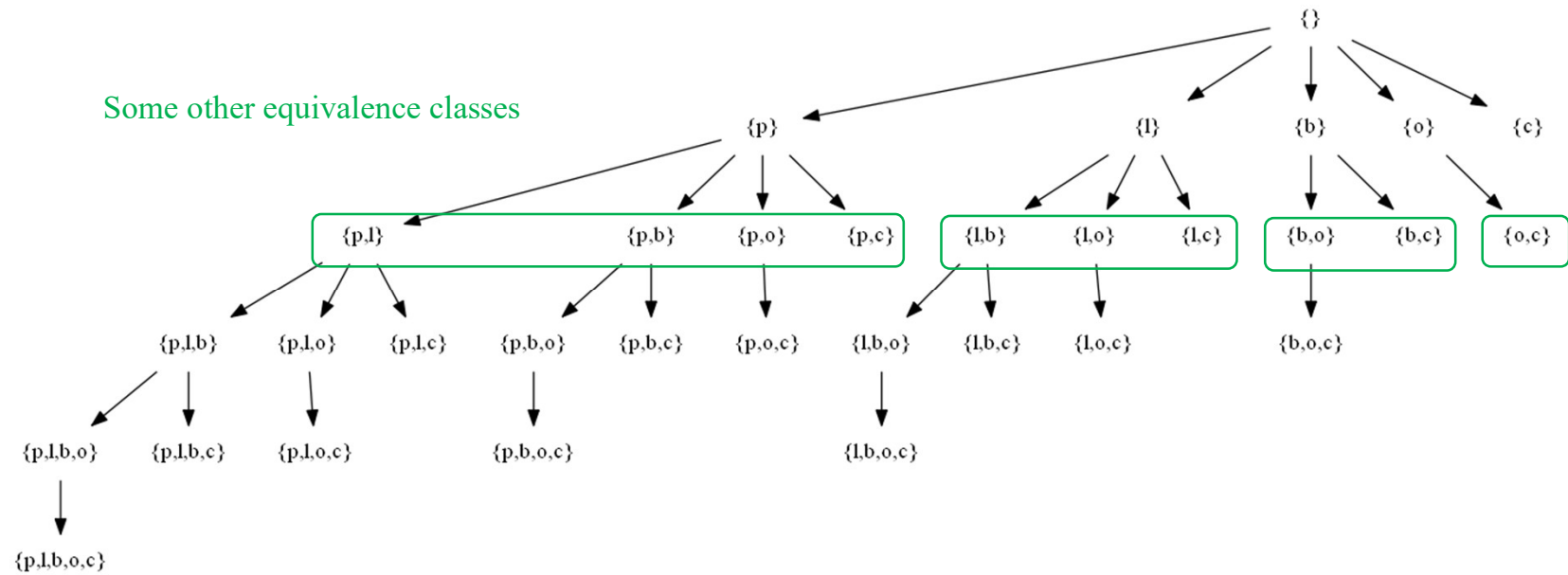
Search space

An equivalence class:



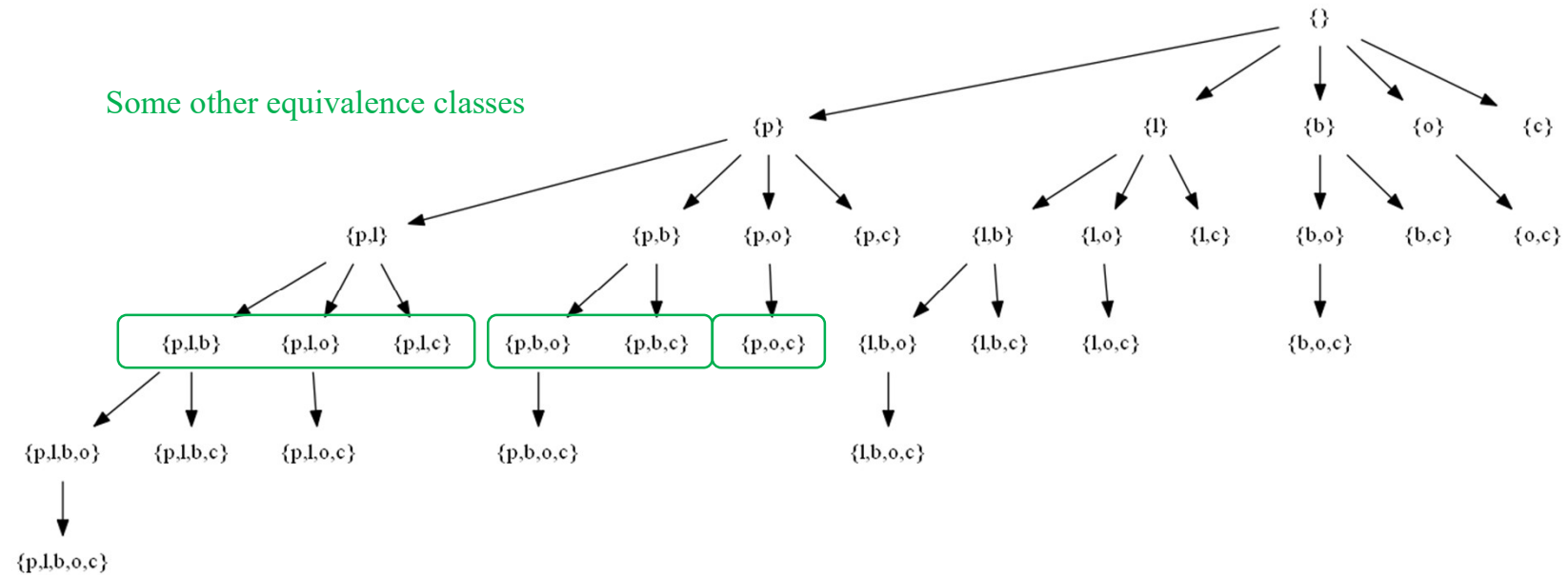
l = lemon
p = pasta
b = bread
o = orange
c = cake

Search space



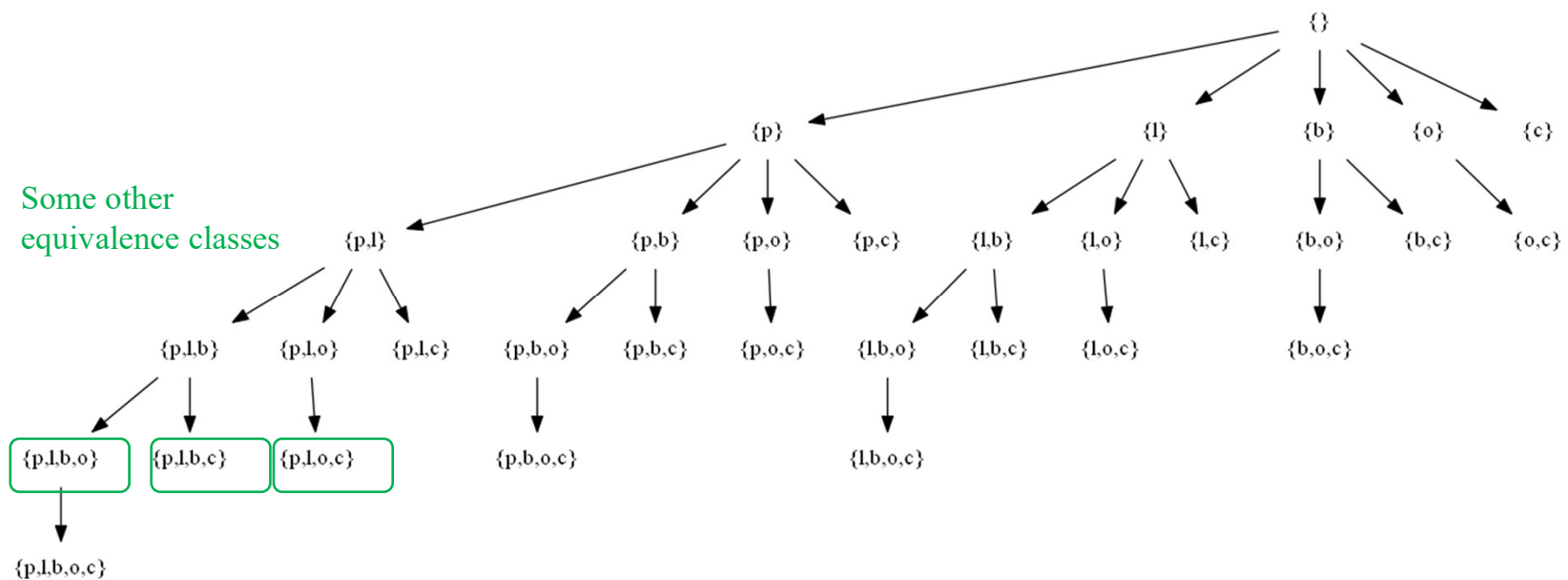
l = lemon
p = pasta
b = bread
o = orange
c = cake

Search space



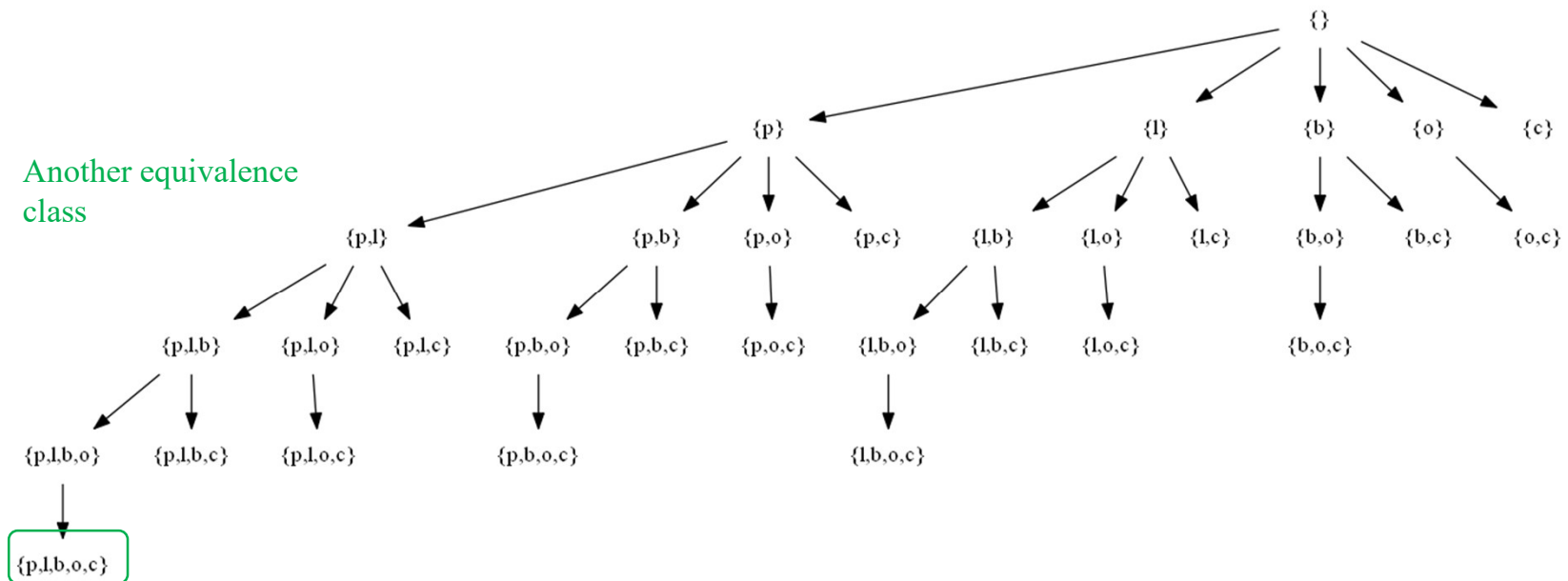
l = lemon
p = pasta
b = bread
o = orange
c = cake

Search space



l = lemon
p = pasta
b = bread
o = orange
c = cake

Search space



l = lemon
p = pasta
b = bread
o = orange
c = cake

The ECLAT algorithm

Step 1: Scan the database to create a vertical representation of the database.



Transaction	Items appearing in the transaction
T1	{ <u>pasta</u> , <u>lemon</u> , <u>bread</u> , orange}
T2	{ <u>pasta</u> , <u>lemon</u> }
T3	{ <u>pasta</u> , orange, cake}
T4	{ <u>pasta</u> , <u>lemon</u> , orange, cake}

The ECLAT algorithm

Step 1: Scan the database to create a vertical representation of the database.

Transaction	Items appearing in the transaction
T1	{ <u>pasta</u> , <u>lemon</u> , <u>bread</u> , orange}
T2	{ <u>pasta</u> , <u>lemon</u> }
T3	{ <u>pasta</u> , orange, cake}
T4	{ <u>pasta</u> , <u>lemon</u> , orange, cake}

↓

item	transactions containing the item
<u>pasta</u>	T1, T2, T3, T4
<u>lemon</u>	T1, T2
<u>bread</u>	T1
orange	T1, T3, T4, T4
cake	T3, T4

The ECLAT algorithm

Step 1: Scan the database to create a vertical representation of the database.

Transaction	Items appearing in the transaction
T1	{ <u>pasta</u> , <u>lemon</u> , <u>bread</u> , orange}
T2	{ <u>pasta</u> , <u>lemon</u> }
T3	{ <u>pasta</u> , orange, cake}
T4	{ <u>pasta</u> , <u>lemon</u> , orange, cake}



item	transactions containing the item
<u>pasta</u>	T1, T2, T3, T4
<u>lemon</u>	T1, T2
<u>bread</u>	T1
orange	T1, T3, T4
cake	T3, T4

Each line is called a
TID-list (Transaction ID List)

The ECLAT algorithm

Step 2: This is the first equivalence class.

<u>pasta</u>	T1, T2, T3, T4
<u>lemon</u>	T1, T2
<u>bread</u>	T1
orange	T1, T3, T4, T4
cake	T3, T4

The ECLAT algorithm

Step 2: This is the first equivalence class.

<u>pasta</u>	T1, T2, T3, T4
<u>lemon</u>	T1, T2
<u>bread</u>	T1
orange	T1, T3, T4 ,T4
cake	T3, T4

ECLAT eliminates infrequent itemsets (*minsup* = 2)

The ECLAT algorithm

Step 2: This is the first equivalence class.

pasta	T1, T2, T3, T4
lemon	T1, T2
bread	T1
orange	T1, T3, T4, T4
cake	T3, T4

ECLAT eliminates infrequent itemsets (*minsup* = 2)

ECLAT outputs the frequent itemsets with 1 items

{pasta}, {lemon}, {orange}, {cake}

The ECLAT algorithm

Step 3: ECLAT combines itemsets of the equivalence class to generate equivalence classes of size $K+1$

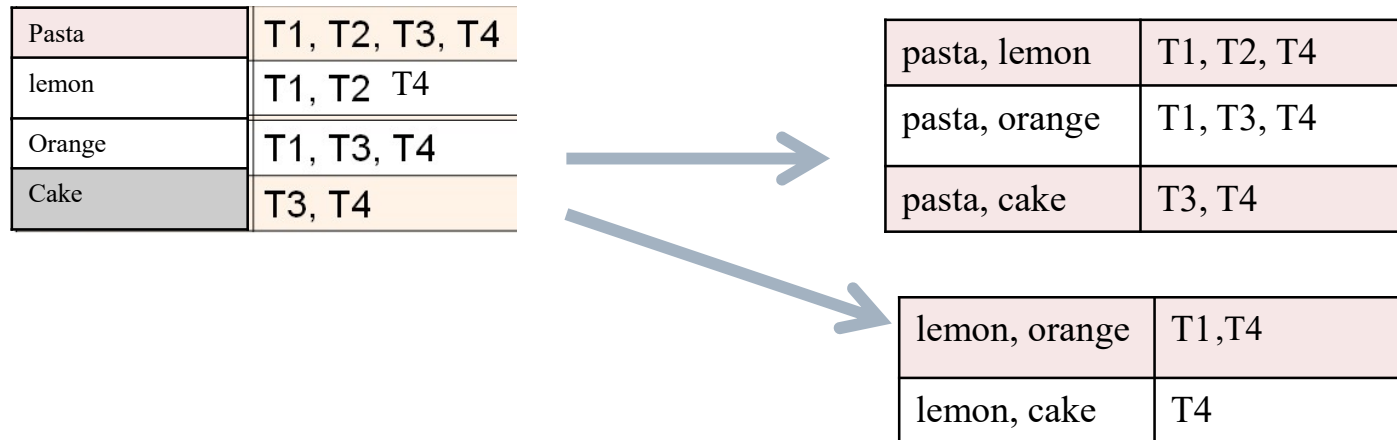
Pasta	T1, T2, T3, T4
lemon	T1, T2 T4
Orange	T1, T3, T4
Cake	T3, T4



pasta, lemon	T1, T2, T4
pasta, orange	T1, T3, T4
pasta, cake	T3, T4

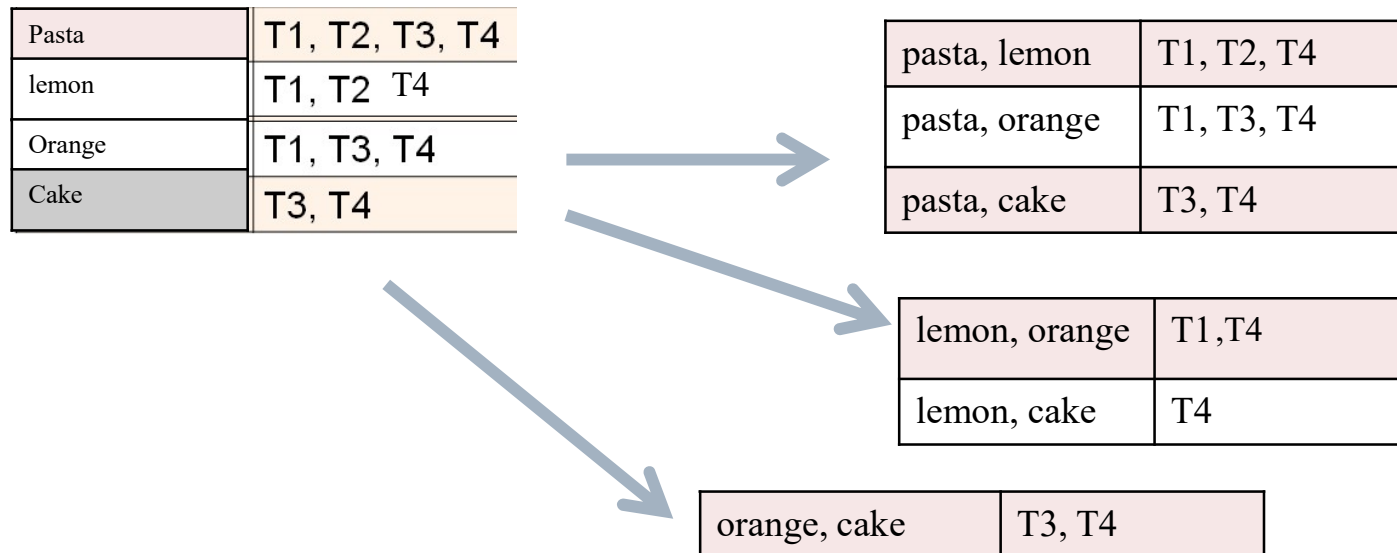
The ECLAT algorithm

Step 3: ECLAT combines itemsets of the equivalence class to generate equivalence classes of size $K+1$



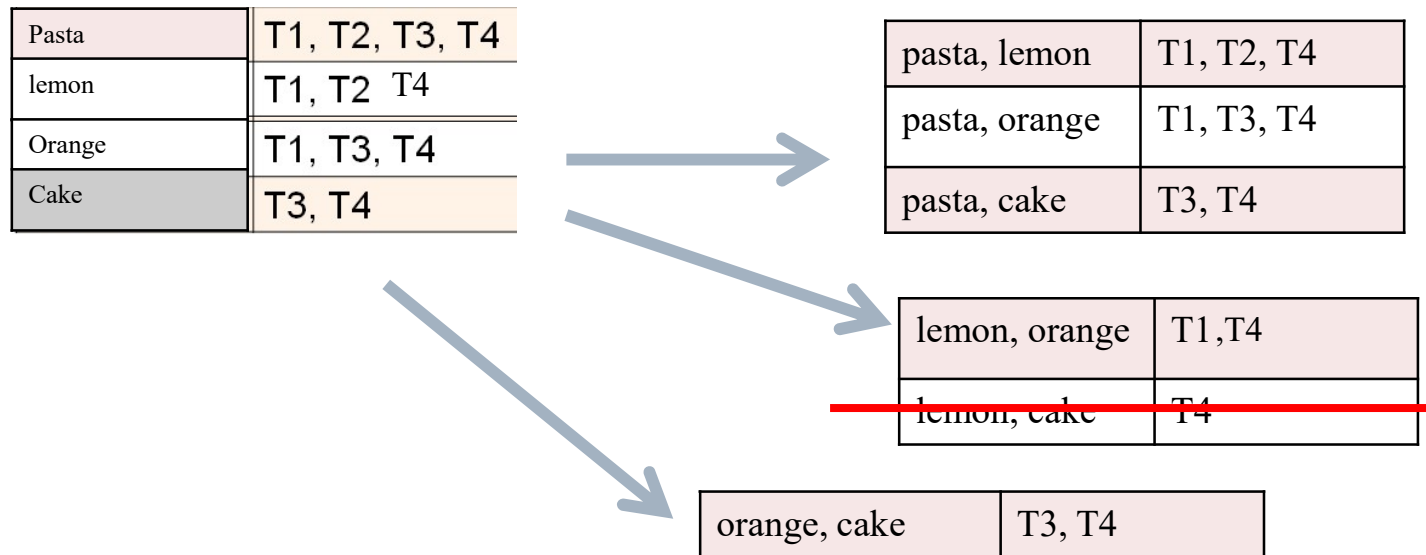
The ECLAT algorithm

Step 3: ECLAT combines combine itemsets of the equivalence class to generate equivalence classes of size $K+1$



The ECLAT algorithm

Step 3: ECLAT combines combine itemsets of the equivalence class to generate equivalence classes of size $K+1$



Then, ECLAT eliminates infrequent itemsets and output the frequent itemsets: $\{\text{pasta, lemon}\}$, $\{\text{pasta, orange}\}$, $\{\text{pasta, cake}\}$, $\{\text{orange, cake}\}$, $\{\text{lemon, orange}\}$

The ECLAT algorithm

Step 4: ECLAT recursively process each equivalence class in the same way.
Consider the first one:

pasta, lemon	T1, T2, T4
pasta, orange	T1, T3, T4
pasta, cake	T3, T4

The ECLAT algorithm

Step 4: ECLAT recursively process each equivalence class in the same way.
Consider the first one:

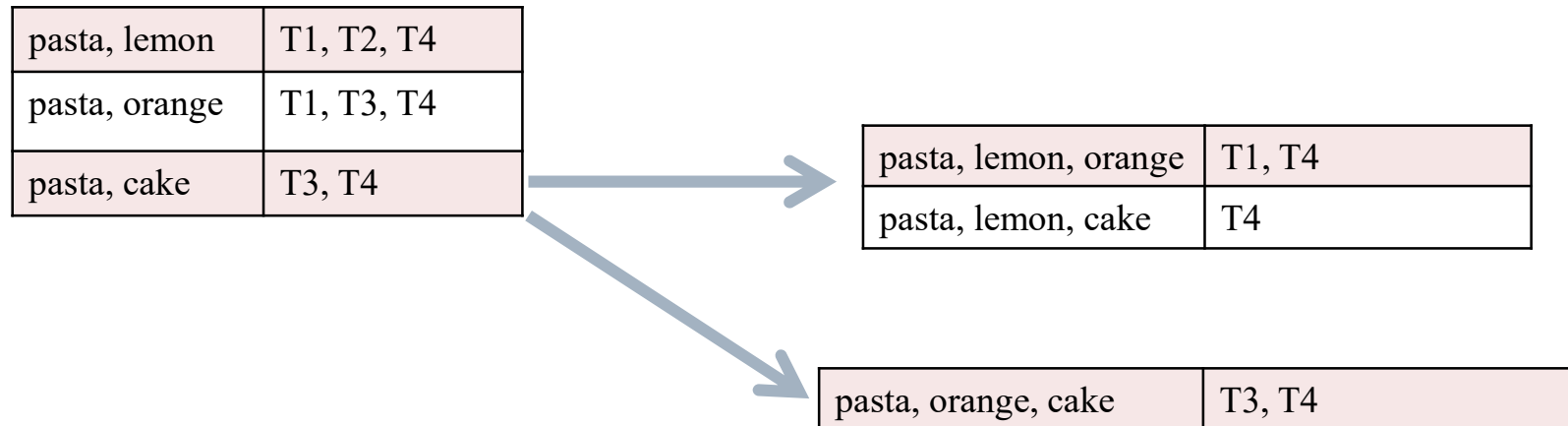
pasta, lemon	T1, T2, T4
pasta, orange	T1, T3, T4
pasta, cake	T3, T4



pasta, lemon, orange	T1, T4
pasta, lemon, cake	T4

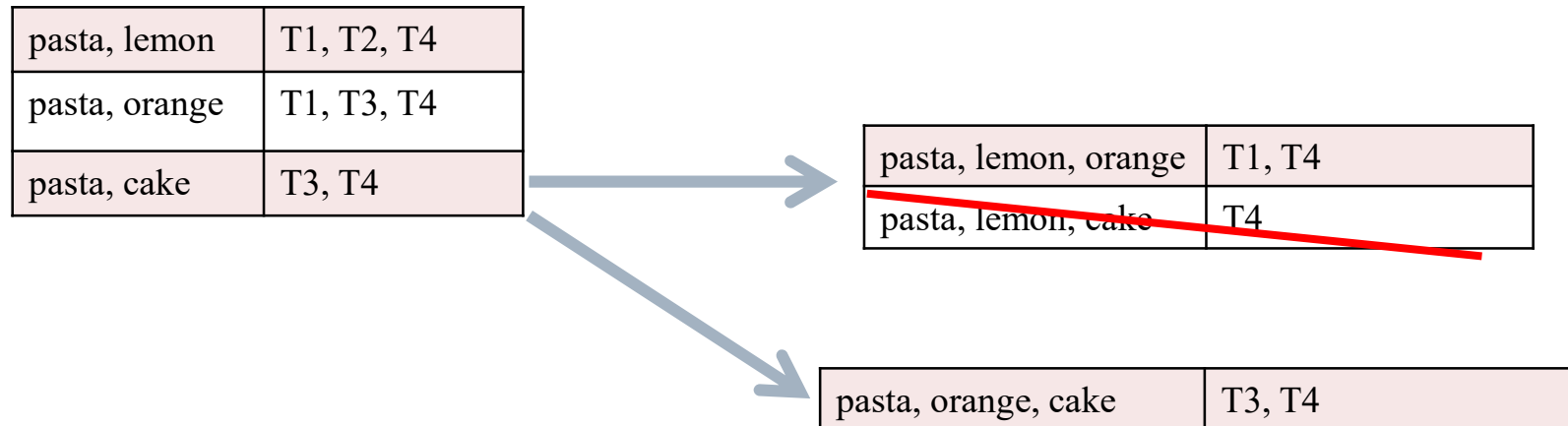
The ECLAT algorithm

Step 4: ECLAT recursively process each equivalence class in the same way.
Consider the first one:



The ECLAT algorithm

Step 4: ECLAT recursively process each equivalence class in the same way. Consider the first one:



ECLAT eliminates infrequent itemsets and output the frequent itemsets

$\{\text{pasta, orange, cake}\}$ $\{\text{pasta, lemon, orange}\}$

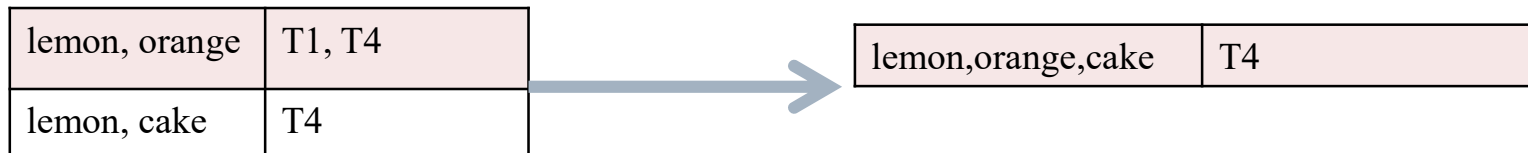
The ECLAT algorithm

Step 4: ECLAT recursively process each equivalence class in the same way.
Consider the next equivalence class:

lemon, orange	T1, T4
lemon, cake	T4

The ECLAT algorithm

Step 4: ECLAT recursively process each equivalence class in the same way.
Consider the next equivalence class:



The ECLAT algorithm

Step 4: ECLAT recursively process each equivalence class in the same way.
Consider the next equivalence class:

lemon, orange	T1, T4
lemon, cake	T4



lemon, orange, cake	T4
--------------------------------	---------------

This itemset is infrequent.
It is eliminated.

The ECLAT algorithm

All other equivalence classes contain a single itemset. Thus, no candidates can be generated.

Final result:

{pasta}	support = 4
{lemon}	support = 3
{orange}	support = 3
{cake}	support = 2
{pasta, lemon}	support: 3
{pasta, orange}	support: 3
{pasta, cake}	support: 2
{lemon, orange}	support: 2
{orange, cake}	support: 2
{pasta, lemon, orange}	support: 2
{pasta, orange, cake}	support: 2

Performance

- In this example:
 - **ECLAT** has explored 14 itemsets.
 - **Apriori** would have explored 18 itemsets.
- How is the performance of ECLAT?
 - ECLAT scans the database a single time to create a vertical database.
 - Then, the most costly operation is the intersection of TID-lists.
 - In the worst case, these lists have the size of the database.
 - **Several possible optimizations→**

Optimization 1: total order

- Does the choice of a total order $<$ influence the performance?

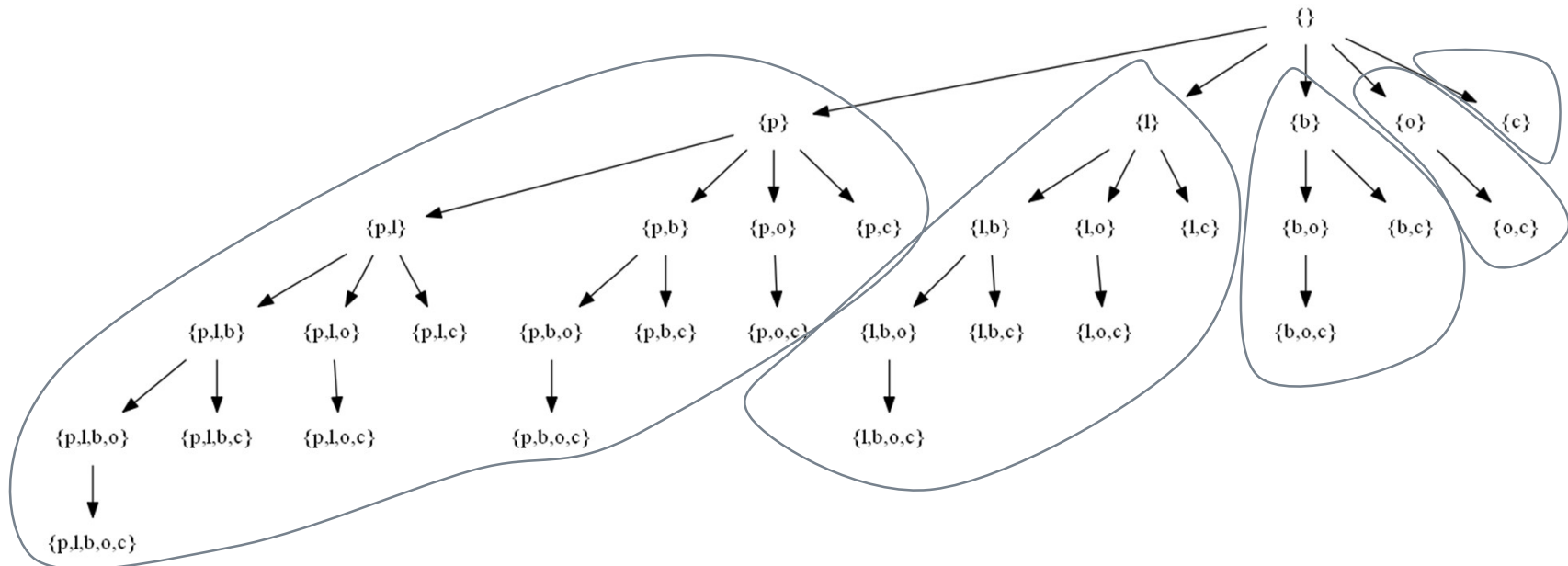
Optimization 1: total order

- ☐ Does the choice of a total order $<$ influence the performance?
- ☐ Yes, but which one to choose?
 - The alphabetical order?

Optimization 1: total order

- ☐ Does the choice of a total order $<$ influences the performance?
- ☐ Yes, but which one to choose?
 - The alphabetical order?
- ☐ A better choice: the order of increasing support.

Observation



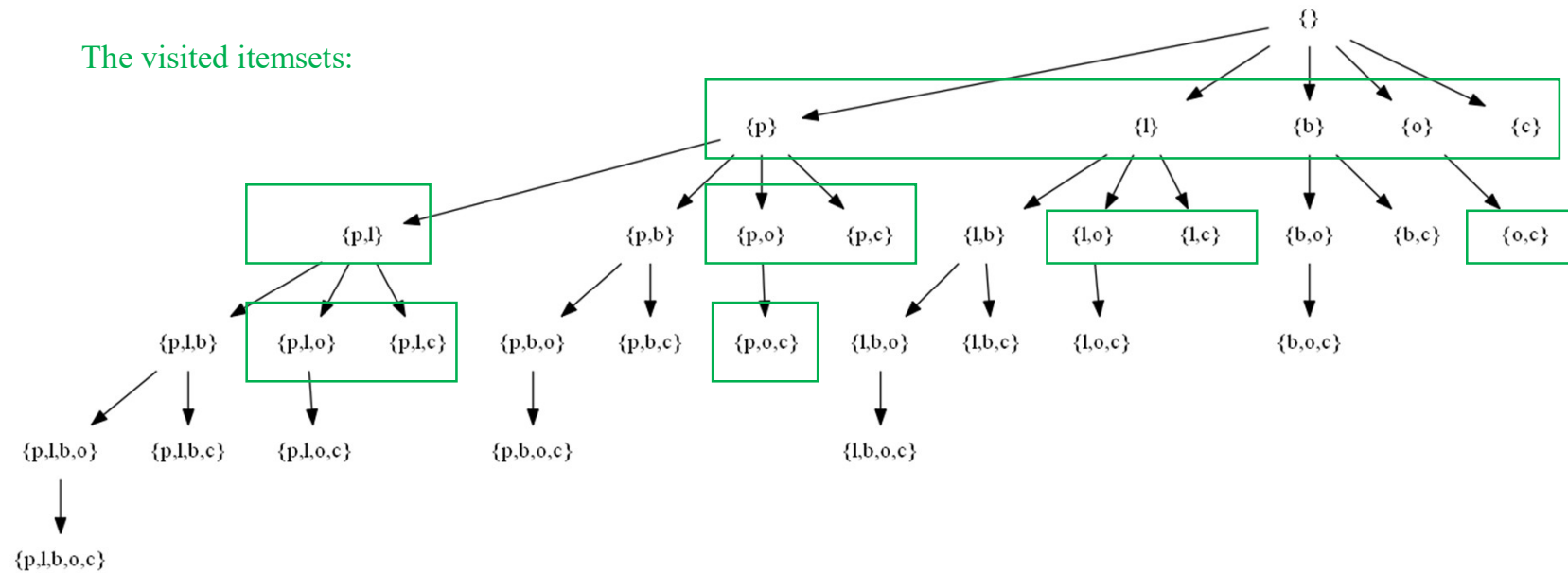
l = lemon
p = pasta
b = bread
o = orange
c = cake

If an item is smaller according to the total order, its sub-tree will be larger.

Search space

pasta < lemon < bread < orange < cake

The visited itemsets:



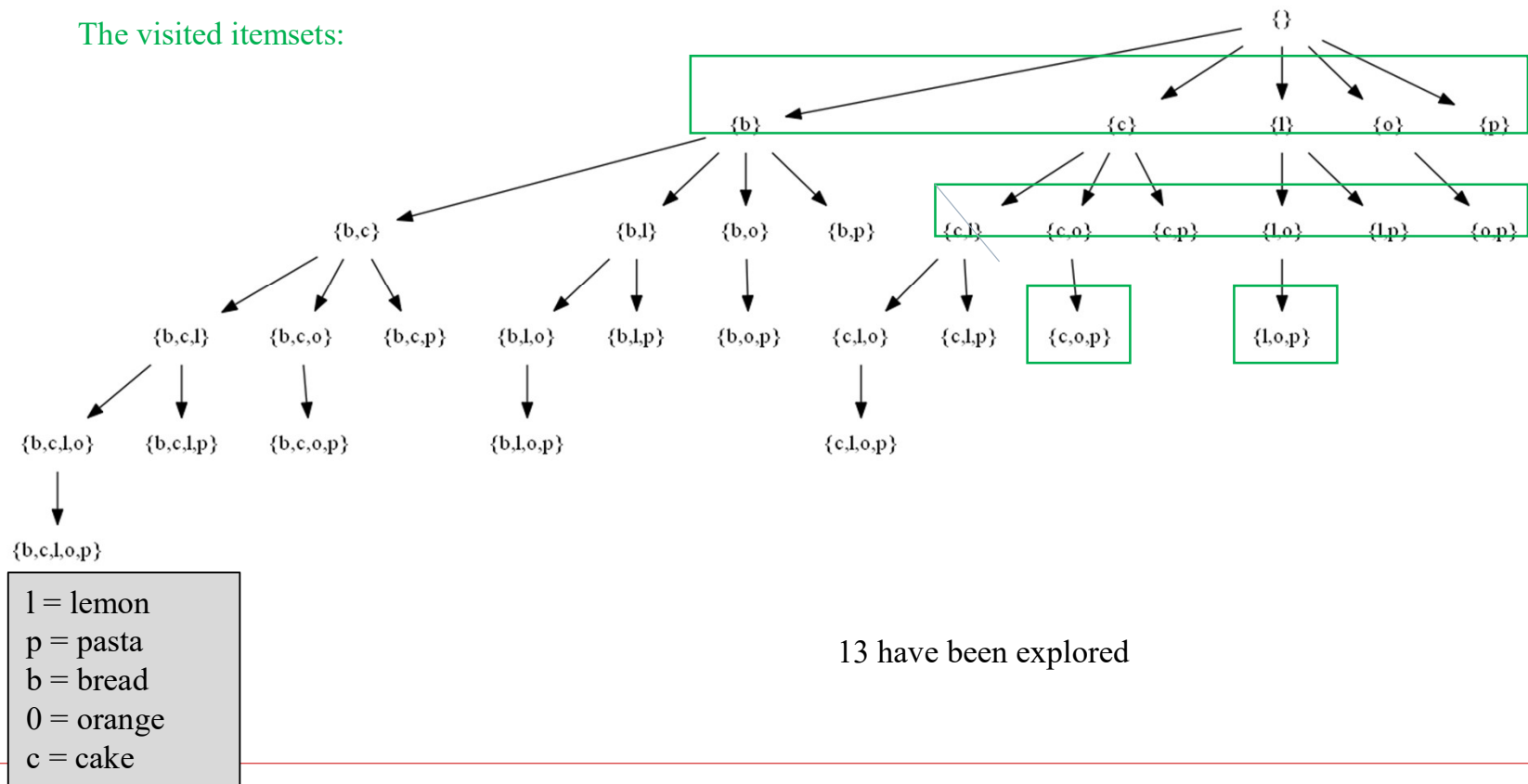
l = lemon
p = pasta
b = bread
o = orange
c = cake

14 have been explored

Search space

bread < cake < lemon < orange < pasta

The visited itemsets:



Optimization 2 - intersection

How to reduce the cost of intersections?

- Utilize bit vectors the represent the lists of transaction ids
- Will be fast if the number of 1 is large compared to the number of zeros

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}



item	transactions containing the item
pasta	1111 (representing T1, T2, T3, T4)
lemon	1101
bread	1000
orange	1011
cake	0011

item	transactions containing the item
pasta	1111
lemon	1101
bread	1000
orange	1011
cake	0011

Example: Calculate the support of {pasta, lemon} :

$$\begin{aligned}
 &\text{transactions}(\{\text{pasta}\}) \cap \text{transactions}(\{\text{lemon}\}) \\
 &= 1111 \text{ LOGICAL_AND } 1101 \\
 &= 1101
 \end{aligned}$$

Thus {pasta, lemon} has a support of 3

Optimization 3 - memory

Consider an equivalence class:

$\{ABCD, ABCE, ABCF, ABCG, ABCH\}$

It can be stored more efficiently as:

$P = \{ABC\}$

$E = \{E, F, G, H\}$

Pseudocode of ECLAT

```
ECLAT(an equivalence class  $C$ )
FOR EACH  $X \in C$ 
     $T = \emptyset$ 
    FOR EACH  $Y \in C$  such that  $X < Y$ 
         $R = X \cup Y$ 
         $t(R) = t(X) \cap t(Y)$ 
        IF  $sup(R) \geq minsup$ 
            THEN Output R
             $T = T \cup \{R\}$ 
    END FOR
    IF  $T \neq \emptyset$  THEN ECLAT( $T$ )
END FOR
```

Exercise

Let a transaction database as:
Apply **ECLAT** algorithm to find all frequent itemsets with **minsupp = 22%**

Tid	Items
100	M1, M2, M5
200	M2, M4
300	M2, M3
400	M1, M2, M4
500	M1, M3
600	M2, M3
700	M1, M3
800	M1, M2, M3, M5
900	M1, M2, M3